Test de Permutación Walk-Forward aplicado al S&P 500

Code

Evaluación de significancia estadística de una estrategia con XGBoost

AUTHOR
Reynaldo Lazcano Armenta, Juan Pablo Cordero Mayorga y Jerónimo Deli Larios — ITAM, Simulación

PUBLISHED
May 14, 2026

1 Introducción al problema

En este reporte se analiza una pregunta importante dentro de las finanzas cuantitativas. **¿Una estrategia de trading basada en *machine learning* realmente encuentra una señal útil en el mercado? ¿O sus buenos resultados históricos podrían explicarse simplemente por azar?** Esta duda es relevante porque cuando se prueban muchos modelos, parámetros o configuraciones distintas, es común encontrar alguna estrategia que parece funcionar bien en datos pasados. Sin embargo, eso no significa que tenga capacidad predictiva real.

Para estudiar este problema se entrenó un clasificador binario con **XGBoost**. El objetivo fue predecir si el índice **S&P 500** subiría o bajaría al día siguiente. La estrategia se evaluó mediante un **backtest walk-forward anclado**. De esta forma, el modelo se entrenó únicamente con la información disponible hasta cada momento y se evitó usar datos futuros.

Después se aplicó un **test de permutación tipo Aronson**. Este método permite generar mercados sintéticos al alterar la estructura temporal original, pero conservando algunas propiedades estadísticas de las barras OHLC. Con esos mercados simulados se construyó una **distribución nula** del Sharpe Ratio. Luego, esta distribución se comparó con el Sharpe obtenido en los datos reales.

El objetivo del trabajo no es encontrar la estrategia más rentable. Lo que se busca es evaluar si el desempeño observado puede considerarse diferente al azar desde un punto de vista estadístico. Esta diferencia es fundamental. Muchas estrategias que parecen exitosas en un backtest terminan siendo producto del sobreajuste. Es decir, funcionan bien en el pasado, pero no necesariamente capturan una relación real ni repetible en el mercado.

2 Marco teórico

2.1 El problema del backtest

Un **backtest** es una simulación histórica de cómo le habría ido a una estrategia de trading en el pasado. La idea suena razonable: si funcionó ayer, quizás funcione mañana. El problema es que el pasado es **un sólo camino realizado**, y casi cualquier modelo suficientemente flexible puede ajustarse a él.

Esto se conoce como **sobreajuste** *(overfitting)*: el modelo aprende los “accidentes” históricos del precio en vez de patrones genuinos.

Una forma de pensarlo: si yo pruebo 100 estrategias completamente aleatorias sobre los mismos datos, **la mejor de las 100 se va a ver fantástica** sólo por suerte. Si el analista únicamente reporta esa, estará presentando ruido como si fuera señal.

2.2 Validación walk-forward

En *machine learning* tradicional es común dividir los datos de forma aleatoria en entrenamiento y prueba. En series de tiempo financieras esto no es adecuado. Si se entrena el modelo con datos de 2018 y luego se prueba con datos de 2010, se estaría usando información del futuro para predecir el pasado. A esto se le conoce como *look-ahead bias*.

Por esta razón se utiliza el enfoque **walk-forward**. En este método el tiempo siempre avanza en orden. Primero se entrena el modelo con los datos disponibles hasta el día t. Después se predice el bloque siguiente $[t, t+\Delta]$. Luego se vuelve a entrenar el modelo con los datos disponibles hasta $t+\Delta$ y se predice el siguiente bloque. Este proceso se repite hasta cubrir todo el periodo de evaluación.

En este trabajo se utiliza la variante **anclada** o *anchored*. Esto significa que la ventana de entrenamiento siempre inicia en el primer día de la muestra y va creciendo con el tiempo. En cada punto t, el modelo se vuelve a entrenar usando toda la información histórica disponible hasta ese momento.

2.3 Datos OHLCV

Trabajamos con **barras diarias** del S&P 500. Cada fila representa un día de operación bursátil y trae cinco números:

Columna	Significado
Open	precio al que abrió el mercado
High	precio máximo del día
Low	precio mínimo del día
Close	precio al que cerró el mercado
Volume	cantidad de acciones operadas en el día

Estas son nuestras **velas**.

En todo el reporte trabajamos con **logaritmo de precios** y **log-retornos** en lugar de precios crudos. Las dos razones principales:

1. **Aditividad.** Los log-retornos suman: el log-retorno de un periodo largo es la suma de los log-retornos diarios. Esto facilita reconstruir trayectorias.
2. **Mejor comportamiento estadístico.** Los retornos diarios son aproximadamente simétricos en torno a cero y se aproximan al porcentaje de cambio para movimientos pequeños:

$$r_t = \log\left(\frac{C_t}{C_{t-1}}\right) \approx \frac{C_t - C_{t-1}}{C_{t-1}}.$$

2.4 Feature engineering

El modelo XGBoost no observa los precios directamente. Le damos **características** *(features)* que resumen la historia reciente:

Feature	Cálculo
`ret_lag_k`	log-retorno de hace k días, $k = 1, \ldots, 5$
`vol_5`	desviación estándar de los retornos de los últimos 5 días
`vol_21`	desviación estándar de los retornos de los últimos 21 días
`rsi_14`	RSI de 14 periodos (indicador de sobrecompra/sobreventa)
`ma_ratio`	$\mathrm{MA}_5/\mathrm{MA}_{21} - 1$ (cruce de medias móviles)

La **variable objetivo** y_t es el signo del retorno del día siguiente:

$$y_t = \mathrm{signo}(r_{t+1}) \in \{-1, +1\}.$$

Y la **señal de trading** del modelo se traduce directamente al P&L:

$$\mathrm{PnL}_t = \mathrm{señal}_t \cdot r_{t+1}, \quad \mathrm{señal}_t \in \{-1, +1\}.$$

Si el modelo predice "subirá" nos ponemos largos (+1) y ganamos r_{t+1}; si predice "bajará" nos ponemos cortos (-1) y ganamos $-r_{t+1}$.

2.5 XGBoost en pocas palabras

XGBoost *(eXtreme Gradient Boosting)* es un algoritmo basado en **árboles de decisión con boosting**. La idea, intuitivamente, es:

1. Se entrena un primer árbol simple.
2. Se identifican los errores de ese árbol.
3. Se entrena un segundo árbol que intenta **corregir** esos errores.
4. Se repite, sumando pequeños árboles correctores hasta que el modelo conjunto sea bueno.

Lo usamos como **clasificador binario**: la salida es la probabilidad de que el mercado suba mañana. Si esa probabilidad es ≥ 0.5 emitimos señal $+1$ (largo), si no, -1 (corto). Para evitar sobreajuste usamos hiperparámetros conservadores y **fijos en todos los folds y permutaciones**: si afináramos los parámetros en cada fold inflaríamos artificialmente la probabilidad de error tipo I del test.

2.6 Mecánica de trading: ¿qué hace el modelo en la práctica?

Es importante distinguir dos momentos completamente separados en la vida del modelo: el **entrenamiento** y la **ejecución OOS**. Tienen objetivos distintos y usan la información de forma diferente.

2.6.1 Durante el entrenamiento: el modelo aprende signos

En cada fold del walk-forward, el modelo recibe las features X_t y aprende a predecir el **signo** del retorno del día siguiente:

$$y_t = \text{signo}(r_{t+1}) \in \{-1,\ +1\}$$

El modelo **nunca ve retornos directamente** — solo ve si el mercado subió o bajó. El algoritmo XGBoost ajusta sus parámetros internos minimizando los errores de clasificación: ¿predije $+1$ cuando debía ser -1? Eso es un error. ¿Predije -1 cuando el mercado sí bajó? Eso es un acierto. El entrenamiento es un problema de clasificación puro.

2.6.2 Durante la ejecución OOS: del clasificador al P&L

Una vez que el modelo está entrenado y se aplica en el periodo fuera de muestra, **ya no hay aprendizaje**. Lo que ocurre es:

1. Se observan las features X_t con datos disponibles al cierre del día t.
2. El modelo emite una señal $\hat{s}_t \in \{-1,\ +1\}$.
3. Se ejecuta la posición: largo si $\hat{s}_t = +1$, corto si $\hat{s}_t = -1$.
4. Al cierre de $t+1$ se liquida la posición y se registra el retorno de la estrategia:

$$r_t^{\text{strat}} = \hat{s}_t \times r_{t+1}^{\text{mercado}}$$

Aquí está el cambio de lenguaje clave: ya no comparamos la señal con un signo para ver si acertamos — la **multiplicamos por el retorno real** para saber cuánto ganamos o perdimos. La tabla siguiente resume la diferencia:

	Entrenamiento	**Ejecución OOS**
La señal es	$+1$ o -1	$+1$ o -1
Se compara con	$\text{signo}(r_{t+1})$ para aprender	—
Se multiplica por	—	r_{t+1} para calcular P&L
Resultado	actualización de pesos	retorno del día

La razón por la que esto funciona aritméticamente es simple: si el modelo acertó el signo, $\hat{s}_t$ y r_{t+1} tienen el mismo signo, por lo que su producto es **positivo**. Si se equivocó, tienen signos opuestos y el producto es **negativo**. Así, la curva de equity acumulada refleja directamente la calidad predictiva del clasificador, expresada en unidades financieras (retornos porcentuales).

2.6.3 El ciclo completo cada día

```
Cierre de t:
  features(X_t)  →  modelo  →  señal +1 o -1  →  ejecuta orden

Cierre de t+1:
  liquida posición  →  PnL_t = señal_t × r_{t+1}  →  registra retorno

Repite al día siguiente.
```

2.6.4 ¿Por qué sufre la estrategia en mercados alcistas?

El S&P 500 tiene un sesgo alcista histórico de ~10% anual. Cada vez que el modelo emite -1 en un día que el mercado sube, no solo deja de ganar sino que *pierde activamente* por ir corto. Una estrategia *long-only*

(siempre $+1$, sin modelo) habría capturado todo el rally de 2005–2024 sin ningún esfuerzo predictivo. Esto no significa que el modelo esté mal programado: significa que las features técnicas disponibles en OHLCV público **no tienen suficiente poder predictivo** para justificar los riesgos de ir corto en uno de los mercados más eficientes del mundo.

2.7 Ratio de Sharpe

El **Sharpe Ratio** mide cuánto rendimiento se obtiene por cada unidad de riesgo (volatilidad):

$$\text{Sharpe} = \frac{\bar{r}}{\sigma}.$$

Para anualizarlo (convertir de diario a anual) multiplicamos por la raíz del número de periodos por año. En finanzas se usa 252, que es aproximadamente el número de días bursátiles en un año:

$$\boxed{\text{Sharpe}_{\text{anualizado}} = \sqrt{252}\,\frac{\bar{r}_{\text{diario}}}{\sigma_{\text{diario}}}}$$

Como referencia rápida:

Sharpe anualizado	Interpretación
< 0	pierde dinero en promedio
$0 - 0.5$	bajo, probablemente no vale la pena
$0.5 - 1.0$	aceptable para una estrategia real
$1.0 - 2.0$	bueno, difícil de lograr de manera consistente
> 2.0	excelente (sospechoso si sólo aparece en backtesting)

Usamos el Sharpe como **estadístico de prueba** del test de permutación.

2.8 Test de permutación de Aronson

El test de permutación es una técnica no paramétrica que **construye la distribución nula del estadístico de manera empírica**, sin asumir ninguna distribución teórica. La lógica:

1. Calculamos el estadístico sobre los datos reales: T_{real}.
2. **Destruimos** cualquier estructura temporal de los datos mediante permutaciones.
3. Calculamos el estadístico sobre cada conjunto permutado: $T_{\text{perm}}^{(i)}, i = 1, \ldots, n$.
4. La distribución empírica de los $T_{\text{perm}}^{(i)}$ es la **distribución nula**.
5. El **p-valor** es la fracción de veces que el estadístico permutado fue igual o mayor al real.

Las hipótesis del test son:

- H_0: el modelo no captura una señal predictiva real; cualquier desempeño observado puede explicarse por azar bajo una trayectoria de mercado sin estructura temporal explotable.

- H_1: el modelo obtiene un desempeño fuera de muestra suficientemente alto como para ser poco probable bajo H_0, lo que sugeriría la presencia de señal predictiva.

El truco es **cómo se permuta**. No se pueden simplemente revolver los precios de cierre, porque eso destruye la distribución de los retornos individuales: la volatilidad cambia, los gaps cambian, los rangos intradía cambian. Aronson propone descomponer cada barra OHLC en componentes aditivos en log-espacio y permutar **cada componente por separado**.

2.9 Descomposición y recomposición de barras en log-espacio

Cada barra diaria se descompone en cuatro componentes independientes en log-espacio:

$$\begin{aligned}
\text{gap}_t &= \log O_t - \log C_{t-1} && \text{(apertura vs. cierre del día previo)} \\
\text{body}_t &= \log C_t - \log O_t && \text{(cuerpo de la vela: subió o bajó)} \\
\text{high}_t &= \log H_t - \log \max(O_t, C_t) && \text{(sombra superior; siempre } \geq 0) \\
\text{low}_t &= \log \min(O_t, C_t) - \log L_t && \text{(sombra inferior; siempre } \geq 0)
\end{aligned}$$

Trabajar en log-espacio permite que estas piezas se sumen aditivamente y que la trayectoria se pueda reconstruir como una caminata. Una vez permutados los cuatro vectores de manera independiente, **reconstruimos** secuencialmente partiendo del último cierre real del periodo de entrenamiento C_{t-1}:

$$\begin{aligned}
\log O_t &= \log C_{t-1} + \text{gap}_t \\
\log C_t &= \log O_t + \text{body}_t \\
\log H_t &= \max(\log O_t, \log C_t) + \text{high}_t \\
\log L_t &= \min(\log O_t, \log C_t) - \text{low}_t
\end{aligned}$$

El resultado es un “universo paralelo” donde:

- la distribución marginal de retornos, gaps, cuerpos y rangos se conserva,
- la volatilidad típica se mantiene parecida,
- pero **el orden temporal y cualquier patrón explotable se eliminan**.

Por eso esta permutación es más cuidadosa que un *shuffle* ingenuo de los precios de cierre.

2.10 Algoritmo de permutación, paso a paso

1. Definir el inicio del periodo fuera de muestra.
2. **Mantener intacto** el periodo inicial de entrenamiento (la pre-OOS).
3. Descomponer las barras OHLC en componentes logarítmicos.
4. Permutar aleatoriamente los componentes del periodo OOS, **cada uno por separado**.
5. Reconstruir una nueva trayectoria OHLC sintética.
6. Recalcular los features sobre la trayectoria permutada.
7. Correr el mismo walk-forward anclado.
8. Calcular el Sharpe OOS de esa permutación.
9. Repetir muchas veces para construir la distribución nula.

2.11 Estadístico de prueba y cálculo del p-valor

El estadístico es el Sharpe Ratio anualizado de los retornos OOS:

$$T_{\text{real}} = \sqrt{252}\,\frac{\bar{r}_{\text{OOS,real}}}{s_{\text{OOS,real}}}, \qquad T_{\text{perm}}^{(i)} = \sqrt{252}\,\frac{\bar{r}_{\text{OOS,perm}}^{(i)}}{s_{\text{OOS,perm}}^{(i)}}.$$

El p-valor se calcula con la **corrección** $+1$, que evita que un número finito de simulaciones produzca un p-valor exactamente cero (lo cual sería imposible de interpretar):

$$\hat{p} = \frac{1 + \#\{i : T_{\text{perm}}^{(i)} \geq T_{\text{real}}\}}{1 + n}.$$

3 Metodología y simulación

3.1 Configuración del experimento

Los parámetros del experimento se definen aquí. `N_PERMUTATIONS` es la única perilla "computacionalmente costosa": cuanto más alto, más preciso es el p-valor pero más tarda la corrida.

▶ Code

3.2 Descarga de los datos del S&P 500

Descargamos 20 años de barras diarias del S&P 500 desde Yahoo Finance, usando el paquete `quantmod`. Esta es la misma fuente que usa el notebook original en Python.

▶ Code

```
Barras totales: 5032
```

▶ Code

```
Periodo:        2005-01-03  →  2024-12-30
```

3.3 Funciones de soporte (todo en R puro)

A continuación definimos las funciones que replican lo que hacía el notebook de Python. Las dejamos directamente en el reporte para que el documento sea **autocontenido**.

▶ Code

3.4 Vista preliminar de los datos

Antes de meter el modelo, vale la pena ver cómo se ve el log-precio y los log-retornos diarios. Nótese que la volatilidad **no es constante**. Hay periodos calmos y periodos turbulentos (la crisis de 2008, el shock COVID de 2020, etc.).

▶ Code

▶ Code

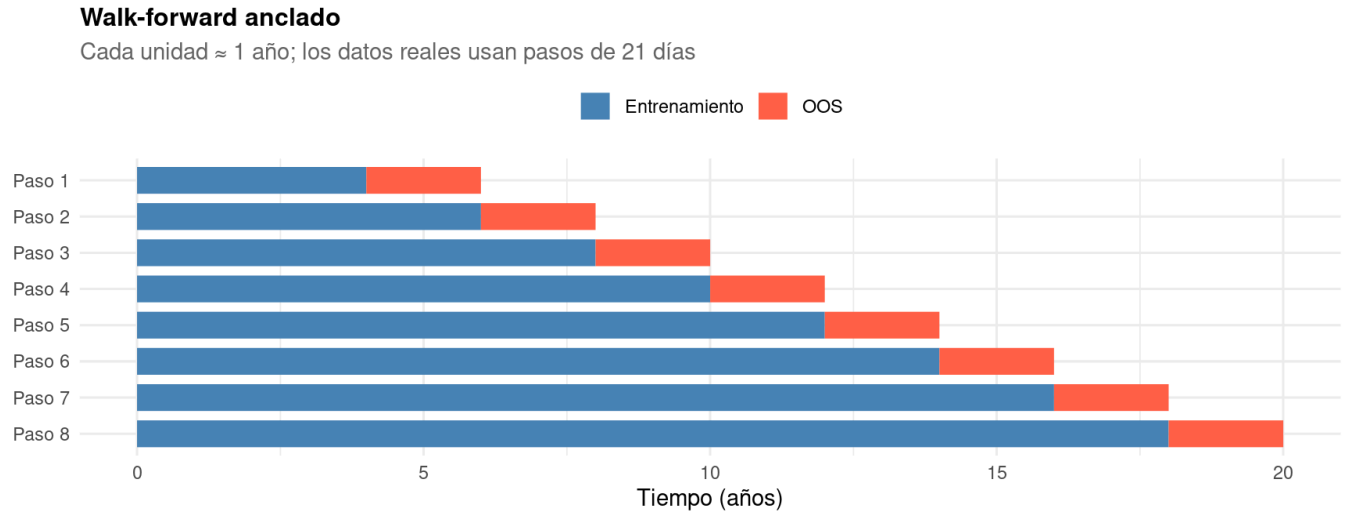
Estadísticas descriptivas de los retornos diarios

metrica	valor
Retorno medio anualizado	0.0797 (7.97%)
Volatilidad anualizada	0.1922 (19.22%)
Sesgo (skewness)	-0.531
Curtosis en exceso	+13.019

3.5 Esquema visual del walk-forward

El siguiente esquema muestra la idea del walk-forward anclado: la ventana de entrenamiento (azul) crece con cada paso y siempre predecimos el bloque inmediato siguiente (naranja).

▶ Code



3.6 Construcción de features y línea base

Construimos los features sobre los datos reales. La salida es la matriz `X_real` (con 9 columnas), el objetivo binario `y_real` y los retornos `r_real` que usaremos para calcular el P&L.

▶ Code

```
Filas de features:     5010
```

▶ Code

```
Número de features:    9
```

▶ Code

```
Features:              ret_lag_1, ret_lag_2, ret_lag_3, ret_lag_4, ret_lag_5, vol_5,
vol_21, rsi_14, ma_ratio
```

▶ Code

```
Primera barra OOS:     2009-02-04
```

▶ Code

```
Pasos de re-entreno:   190
```

▶ Code

```
Sube/baja (% sube):    54.5 %
```

3.7 Walk-forward sobre los datos reales

Corremos el walk-forward anclado una sola vez sobre los datos reales para obtener nuestra línea base T_{real}.

▶ Code

```
Walk-forward real listo en 59.5 s
```

▶ Code

```
Barras OOS:        4002
```

▶ Code

```
Sharpe anualizado: +0.032
```

▶ Code

```
Profit Factor:     1.006
```

▶ Code

```
CAGR:              +0.57%
```

3.8 Test de permutación Monte Carlo

Cada iteración del Monte Carlo es independiente: depende del DataFrame original y de su semilla.

▶ Code

▶ Code

```
Primera fecha OOS:    2009-02-04
```

▶ Code

```
start_idx (OHLC):     1030
```

▶ Code

```
Corriendo 100 permutaciones en 11 workers...
```

▶ Code

```

Tiempo total: 2437.7 s (24377 ms / permutación)
```

▶ Code

```
Sharpe real:                       +0.032
```

▶ Code

```
Permutaciones con Sharpe >= real:  97 / 100
```

▶ Code

```
p-valor (con corrección +1):       0.9703
```

4 Resultados e interpretación

4.1 Tabla resumen

▶ Code

Resumen del test de permutación walk-forward

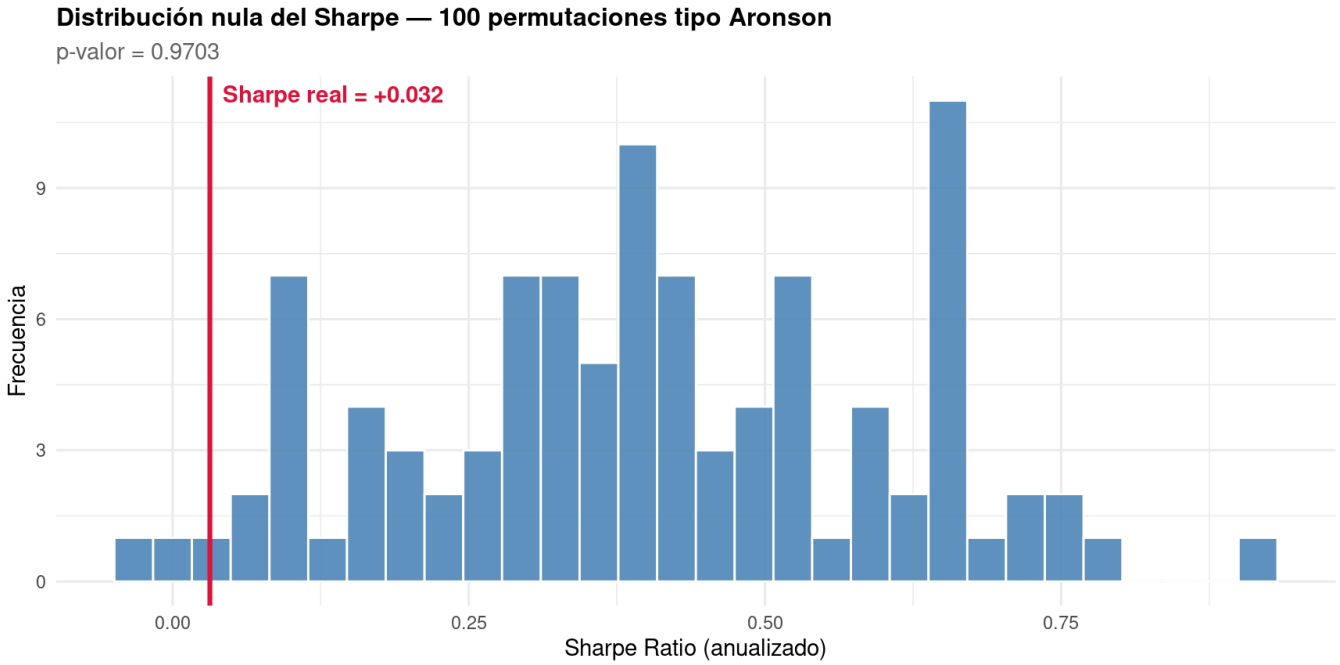
Métrica	Valor
Sharpe real (T_real)	+0.0315
Media de Sharpes permutados	+0.4011
Desv. estándar de la nula	0.2026
Percentil 5% de la nula	+0.0901
Percentil 95% de la nula	+0.7084
# permutaciones con Sharpe ≥ real	97
p-valor (corrección +1)	0.9703
Permutaciones totales	100

4.2 Distribución nula del Sharpe

El gráfico central del test: el histograma azul es la distribución de los Sharpes OOS bajo la hipótesis nula (mercado sin estructura temporal); la línea roja es el Sharpe real obtenido con los datos verdaderos.

- Si la línea roja queda **muy a la derecha** del histograma, el Sharpe real sería raro bajo H_0 → evidencia de señal real.
- Si queda **dentro o a la izquierda** del histograma, el Sharpe real podría haberse obtenido por puro azar → no hay evidencia de señal.

▶ Code



4.3 Curva de equity OOS real

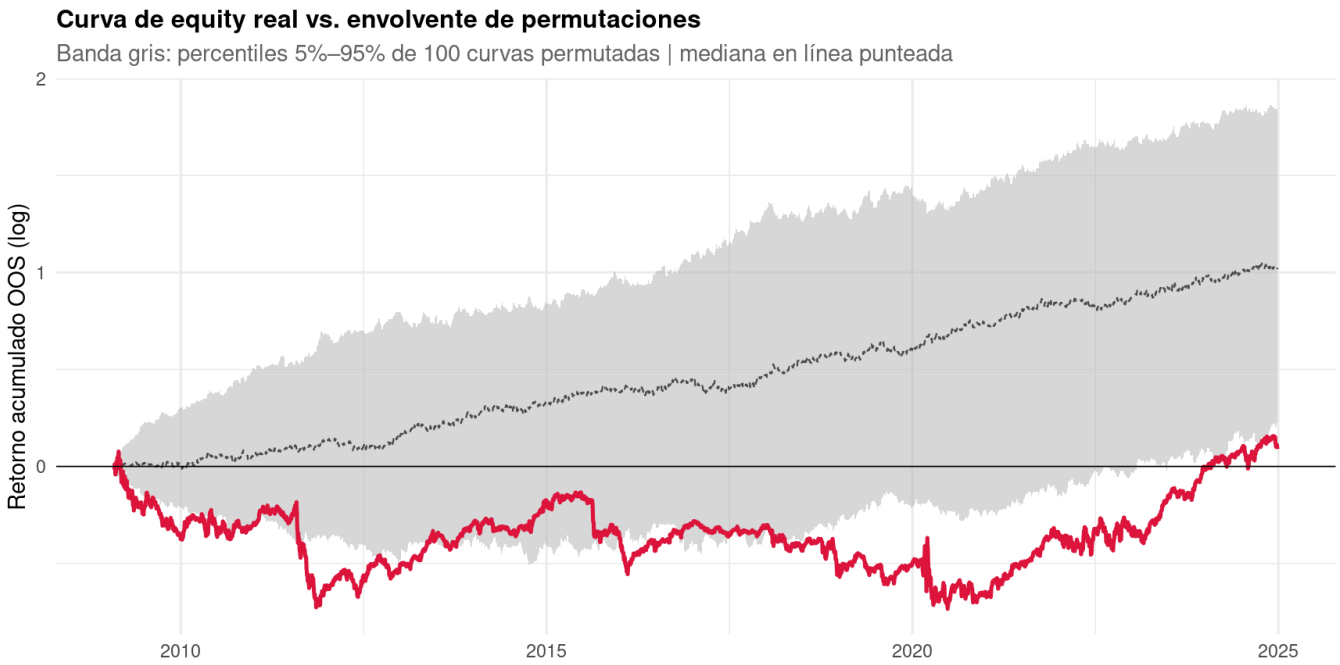
La curva de equity acumula los retornos OOS día a día. Una curva ascendente estable es señal de una estrategia que gana consistentemente; una curva plana o errática es lo opuesto.

▶ Code

4.4 Curva real vs. envolvente de permutaciones

La banda gris muestra el rango entre los percentiles 5% y 95% de las curvas de equity permutadas; la línea negra punteada es la mediana. La estrategia real, en rojo, debería vivir consistentemente arriba de la banda si su desempeño es “real”.

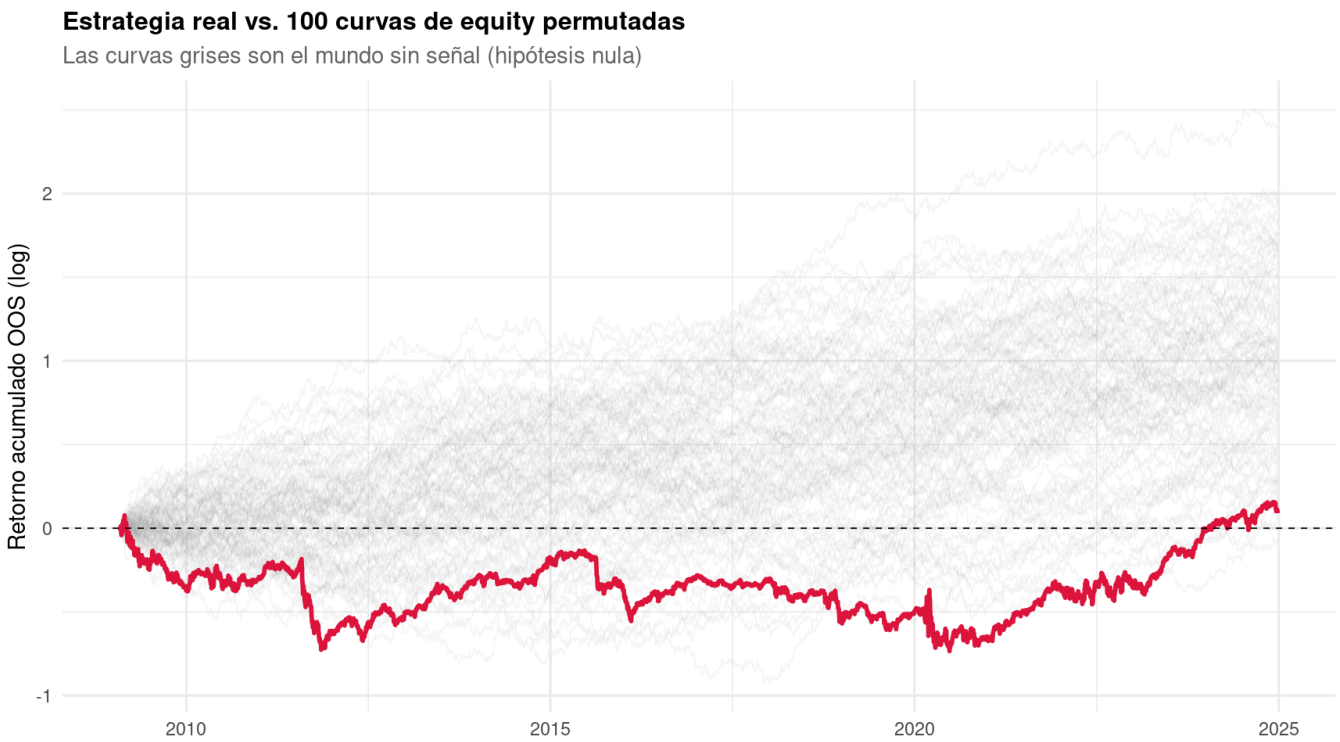
▶ Code



4.5 Todas las curvas permutadas superpuestas

Para tener una vista completa de la “nube” de mundos sin señal, graficamos todas las curvas permutadas en gris muy translúcido junto con la curva real en rojo.

▶ Code



4.6 Interpretación de los resultados

▶ Code

Con un Sharpe real de **+0.032** y un p-valor de **0.9703**, la evidencia es **no significativa** ($p \geq 0.10$).

▶ Code

Esto sugiere que el desempeño OOS del modelo **es indistinguible del azar**: una fracción no despreciable de los mercados sintéticos sin estructura obtuvo Sharpes iguales o mayores que el real. No tenemos evidencia estadística suficiente para concluir que el modelo capture señal real.

▶ Code

Significancia estadística vs. significancia práctica. Aun si el p-valor fuera muy bajo, eso no implicaría que la estrategia sea rentable en la práctica. Falta incorporar:

- comisiones por operación,
- *slippage* (deslizamiento entre el precio de la señal y el de ejecución),
- costo de mantener posiciones cortas,
- impuestos,
- restricciones de liquidez.

Una estrategia con Sharpe OOS de ~0.5 antes de costos puede convertirse en una estrategia con Sharpe negativo después de fricciones reales.

5 Conclusiones

En este reporte aplicamos un **test de permutación walk-forward** para evaluar si una estrategia de trading basada en XGBoost sobre el S&P 500 captura alguna señal predictiva real, o si su desempeño OOS podría explicarse por azar. La metodología tiene tres ingredientes principales:

1. **Walk-forward anclado**: el modelo se reentrena cada 21 días con toda la historia disponible y predice el bloque inmediato siguiente. Esto imita cómo se usaría en producción.
2. **Permutación tipo Aronson** de las barras OHLC en log-espacio: cada componente (gap, body, sombra superior, sombra inferior) se permuta de manera independiente, conservando la distribución marginal de cada uno pero destruyendo el orden temporal.
3. **Distribución nula empírica del Sharpe** construida sobre 100 permutaciones, contra la cual se compara el Sharpe real.

▶ Code

El resultado obtenido fue un Sharpe real de **+0.032** y un p-valor de **0.9703**.

▶ Code

Bajo el umbral convencional de 0.05, **no rechazamos** la hipótesis nula. El desempeño del modelo es estadísticamente compatible con el azar.

▶ Code

Este resultado debe tomarse con cautela: depende del periodo histórico, del número de permutaciones (que en este reporte es relativamente bajo, 100), y del modelo y features específicos elegidos.

5.1 Trabajo futuro

Como extensiones naturales del análisis se pueden considerar:

- **Aumentar el número de permutaciones** a 500 o 1,000 para precisión del p-valor (en el reporte original se usaron 1,000).
- Probar **otros estadísticos de prueba** además del Sharpe: Profit Factor, Calmar Ratio, máxima caída *(max drawdown)*.
- Incluir **costos de transacción y slippage** realistas en el cálculo del PnL.
- Comparar el modelo contra una **estrategia benchmark** simple (compra y retén, momentum sencillo).
- Probar **bootstrap por bloques** para conservar parcialmente la agrupación de volatilidad (clustering de volatilidad), que la permutación Aronson destruye.
- Probar **otros modelos** (Random Forest, redes neuronales, modelos lineales regularizados) y **otros conjuntos de features** (microestructura, flujos de órdenes, sentimiento).

5.2 Limitaciones

- **Periodo histórico único.** El análisis cubre 2005-2024. Otra ventana histórica podría dar otra conclusión.
- **Sin costos de transacción ni slippage.** Los retornos brutos sobreestiman el desempeño real.
- **Decisiones del analista.** Los features y los hiperparámetros fueron fijados antes del experimento, pero no se controlan los grados de libertad empleados al elegirlos en su momento.
- **Pocas permutaciones.** Con 100 permutaciones, el p-valor tiene incertidumbre Monte Carlo no despreciable.
- **La permutación destruye clustering de volatilidad.** Estrategias que dependen de regímenes de volatilidad podrían quedar penalizadas.
- **No es recomendación financiera.** Este es un ejercicio académico; no implica que se deba operar la estrategia.

5.3 Notas de reproducibilidad

- La semilla maestra `RNG_SEED <- 42` controla todo el aleatorismo.
- Las semillas hijas para cada permutación se generan deterministamente mediante `sample.int(...)` después de fijar la semilla maestra.
- Los hiperparámetros de XGBoost están fijos (`xgb_params`) para evitar inflar la tasa de error tipo I por *snooping* sobre los folds.
- La paralelización se maneja con `furrr::future_map` sobre un *plan* `multisession`. Con la opción `furrr_options(seed = TRUE)`, los resultados son reproducibles incluso al cambiar el número de workers.

5.4 Bibliografía

- Aronson, D. (2007). *Evidence-Based Technical Analysis*. Wiley.
- White, H. (2000). “A Reality Check for Data Snooping.” *Econometrica*, 68(5).
- Hansen, P. (2005). “A Test for Superior Predictive Ability.” *Journal of Business & Economic Statistics*, 23(4).

- Bailey, D., Borwein, J., López de Prado, M. y Zhu, Q. (2014). “The Deflated Sharpe Ratio.” *Journal of Portfolio Management*.
- Chen, T. y Guestrin, C. (2016). “XGBoost: A Scalable Tree Boosting System.” *Proceedings of the 22nd ACM SIGKDD*.